Что же происходит когда работает какая-либо программа? Запущенная программ выполняет одну простую вещь - она исполняет инструкции. Каждую секунду миллионы и даже возможно миллиарды инструкций извлекаются процессором из оперативной памяти, декодирует их (например распознает к какому типу принадлежит эта инструкция) и исполняет ее. Это может быть сложение двух чисел, доступ к памяти, проверка условия, переход к функции итд. После окончания выполнения одной инструкции процессор переходит к выполнению другой. И так инструкция за инструкцией исполняются до тех пор, пока программа не завершится.

Данный пример естественно рассмотрен упрощенно - на самом деле для ускорения работы процессора современное железо позволяет исполнять инструкции вне очереди, просчитывать возможные результаты, выполнять инструкции одновременно и тому подобные ухищрения.

Описанная нами упрощенная форма работы похожа на фон-неймановскую модель вычислений. Фон-нейман это один из пионеров компьютерных систем, также он один из авторов теории игр. Во время работы программы происходит еще куча других событий, работает множество других процессов и сторонней логики, основная цель которых - упрощение запуска, работы и обслуживания системы.

Существует набор программного обеспечения, которой ответственен за простоту запуска программ (или даже позволяющий запускать несколько программ одновременно), который позволяет программам делить одну и ту же память, позволяющий программам взаимодействовать с устройствами. Такой набор ПО по сути и называют операционной системой и в его задачу входит отслеживание того что система работает корректно и эффективно, а также обеспечивает простоту управления этой системой.
Операцио́нная систе́ма, сокр. **ОС** — комплекс взаимосвязанных программ, предназначенных для управления ресурсами компьютера и организации взаимодействия с пользователем.

В первую очередь ОС добивается своей эффективности через самую главную технику - технику виртуализации. ОС взаимодействует с физическим ресурсом (процессор, память, диск итп) и трансформирует его в более общую, с большими возможностями и более простую для использования форму самого себя. Поэтому для общего понимания можно очень грубо сравнить операционную систему с виртуальной машиной.

Для того чтобы позволять пользователям давать команды операционной системе и таким образом использовать возможности виртуальной машины (такие как запуск программы, выделение памяти, доступ к файлу итд) операционная система предоставляет некоторый интерфейс, называемый API (application programming interface) и к которому можно делать вызовы (call). Типичная операционная система дает возможность сделать сотни системных вызовов.

И наконец, поскольку виртуализация позволяет множеству программ работать (таким образом совместно использовать CPU), одновременно получать доступ к их инструкциям и данным (таким образом разделяя память), а также получать доступ к дискам (таким образом совместно использовать устройства ввода-вывода), операционную систему еще называют менеджером ресурсов. Каждый процессор, диск и память это

ресурс системы и таким образом одной из ролей операционной системы становится задачу по управлению этими ресурсами, делая это эффективно, честно или наоборот, в зависимости от задачи для которой эта операционная система разработана.

Виртуализация CPU

Рассмотрим следующую программу:
(https://youtu.be/zDwT5fUcki4)

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: cpu <string>\n");
        exit(1);
    }
    char *str = argv[1];

    while (1) {
        Spin(1);
        printf("%s\n", str);
    }
    return 0;
}
```

Она не выполняет каких-то особых действий, по сути всё что она делает - вызывает функцию spin(), задача которой циклическая проверка времени и возврат, после того как прошла одна секунда. Таким образом она повторяет бесконечно строку, которую пользователь передал в качестве аргумента.

Запустим эту программу и передадим ей аргументом символ A. Результат получается не особо интересный - система просто выполняет программу, которая периодически выводит на экран символ A.

Теперь попробуем вариант, когда запущено множество экземпляров одной и той же программы, но выводящие разные буквы, чтобы было понятнее. Результат получился в этом случае несколько иной. Не смотря на то что у нас один процессор, программа вроде как выполняется одновременно. Как же так получается? А получается что операционная система, не без помощи возможностей оборудования конечно создает иллюзию. Иллюзию

того, что в системе есть несколько виртуальных процессоров, превращая один физический процессор в теоретически бесконечное количество и позволяя таким образом программам на вид выполняться одновременно. Такую иллюзию и называют Виртуализацией CPU.

Подобная картина порождает много вопросов, например, если несколько программ желают запуститься одновременно, то какая именно будет запущена? За этот вопрос отвечают политики ОС. Политики используются во многих местах в ОС и отвечают на подобные вопросы и являются базовыми механизмами, которые ОС воплощают. Отсюда и роль ОС как ресурсного менеджера.

Виртуализация памяти

Теперь давайте рассмотрим память. Физическая модель памяти в современных системах представляется как массив байт. Чтобы прочитать из памяти нужно указать адрес ячейки, чтобы получить к ней доступ. Чтобы записать или обновить данные нужно также указать данные и адрес ячейки, куда их записать.

Обращения к памяти происходит постоянно в процессе работы программы. Программа хранит в памяти всю ее структуру данных и обращается к ней выполняя различные инструкции. Инструкции между тем тоже хранятся в памяти, поэтому обращение к ней происходит также на каждый запрос к следующей инструкции.

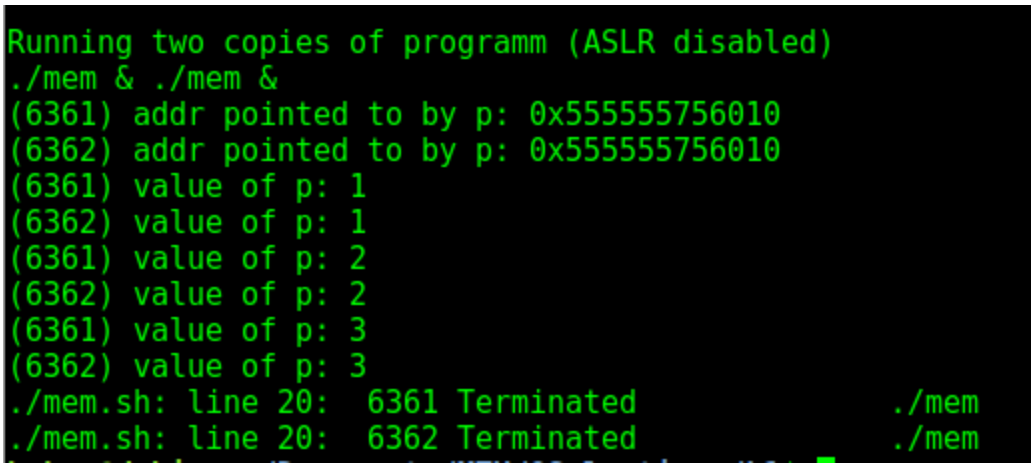
Рассмотрим следующую программу, которая выделяет область памяти, используя вызов malloc():

(https://youtu.be/jnlKRnoT1m0)

```
1  #include <unistd.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "common.h"
5
6  int main(int argc, char *argv[]) {
7      int *p = malloc(sizeof(int));
8      assert(p != NULL);
9      printf("(%d) addr pointed to by p: %p\n", (int) getpid(), p);
10     *p = 0;
11     while (1) {
12 Spin(1);
13 *p = *p + 1;
14 printf("(%d) value of p: %d\n", getpid(), *p);
15     }
16     return 0;
17 }
```

Программа делает несколько вещей. Во-первых, выделяет некоторый объем памяти (строка 7), затем выводит адрес выделенной ячейки (строка 9), записывает ноль в первый

слот выделенной памяти. Затем программ входит в цикл, в котором инкрементирует значение записанное в памяти по адресу в переменной p. Также она выводит идентификатор процесса самого себя. Идентификатор процесса уникален для каждого запущенного процесса. Запустив же несколько копий мы наткнемся на интересный результат: В первом случае, если не сделать ничего и просто запустить несколько копий то адреса будут разными. Но это же не попадает под нашу теорию! Верно, поскольку в современных дистрибутивах включена по-умолчанию функция рандомизации памяти. Если ее отключить, получим ожидаемый результат - адреса памяти у двух одновременно работающих программ будут совпадать.



В итоге получается, что 2 независимых программы работают со своими собственными приватными адресными пространствами, которые в свою очередь операционная система каким-то образом отображает в физической памяти. Поэтому использование адресов памяти в пределах одной программы никак не будет затрагивать другие и каждой программе кажется что у нее собственный кусок физической памяти, целиком отданный в ее распоряжение. Реальность, однако, такова, что физическая память - разделяемый ресурс, управление которым осуществляет операционная система.

Согласованность

Еще одна из важных тем в рамках операционных систем - согласованность. Этот термин используется когда речь идет о проблемах в системе, которые могут возникать при работе с многими вещами одновременно в пределах одной программы. Проблемы согласованности возникают даже в самой операционной системе. В предыдущих примерах с виртуализацией памяти и процессора, ОС управляет многими вещами одновременно - запускает первый процесс, затем второй и так далее. Как оказалось, такое поведение может привести к некоторым проблемам. Так например современные многопоточные программы испытывают такие трудности.

Рассмотрим следующую программу:

```c
#include <stdio.h>
#include <stdlib.h>
#include "common.h"
#include "common_threads.h"

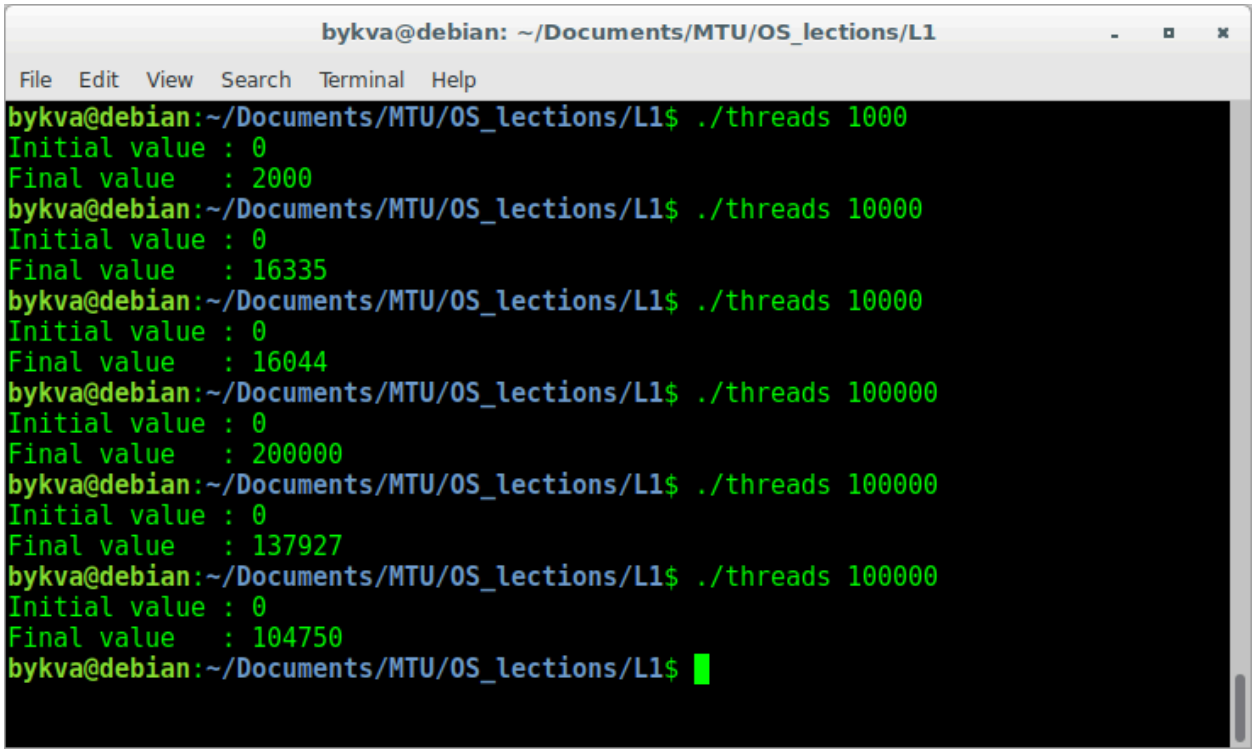
volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
	counter++;
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
	fprintf(stderr, "usage: threads <loops>\n");
	exit(1);
    }
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);
    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value   : %d\n", counter);
    return 0;
}
```

Программа в главной функции создает два потока используя вызов Pthread_create(). В данном примере о потоке можно допустить думать о функции, запущенной в одном пространстве памяти рядом с другими функциями, причем количество запущенных одновременно функций более одной. В данном примере каждый поток стартует и выполняет функцию worker(), которая в свою очередь просто инкрементирует переменную.

Запустим эту программу с аргументом 1000. Как вы уже могли догадаться, результатом должно стать 2000, поскольку каждый поток инкрементировал переменную 1000 раз. Однако все не так просто. Попробуем запустить программу с числом повторений на порядок больше.



Подавая на вход число, например, 100000 мы ожидаем увидеть на выходе число 200000. Однако, запустив с числом 100000 несколько раз, мы не только не увидим правильный ответ, но и разные неправильные ответы. Разгадка кроется в том, что для увеличения числа требуется три операции - извлечение числа из памяти, инкрементация и затем запись числа обратно. Поскольку все эти инструкции не осуществляются атомарно (все одновременно), такие странные вещи могут происходить. Эта проблема называется в программировании race condition - состояние гонки. Когда неизвестные силы в неизвестный момент могут повлиять на выполнение каких-либо ваших операций.

Классификация ОС

С течением развития ОС все возникающие вопросы и способы их решения выливались зачастую в отдельный продукт, каждый из которых занимал собственную нишу.

По назначению

1. Системы общего назначения.

Подразумевает ОС, предназначенные для решения широкого круга задач, включая запуск различных приложений, разработку и отладку программ, работу с сетью и мультимедиа.
2. специализированные системы.
Это различные ОС, ориентированные, прежде всего на эффективное решение определенного класса, с большим или меньшим ущербом для прочих задач

По характеру взаимодействия с пользователем
1. Пакетные ОС, обрабатывающие заранее подготовленные задания
2. Системы разделения времени
3. Системы реального времени

Первые предназначались для решения задач в основном вычислительного характера, не требующих быстрого получения результатов. Критерий создания таких ОС – максимальная пропускная способность при хорошей загрузке всех ресурсов компьютера. В таких системах пользователь отстранен от компьютера.
Системы разделения времени обеспечивают удобство и эффективность работы пользователя, который имеет терминал и может вести диалог со своей программой.
Системы реального времени предназначены для управления техническими объектами (станок, спутник, технологический процесс, например доменный и т.п.), где существует предельное время на выполнение программ, управляющих объектом.

По числу одновременного выполнения задач
1. Однозадачные ОС.
В таких системах в каждый момент времени может существовать не более чем один пользовательский процесс. Однако, одновременно с этим, могут работать системные процессы
2. Многозадачные ОС.
Они обеспечивают параллельное выполнение некоторых пользовательских процессов. Реализация многозадачности требует значительного усложнения алгоритмов и структур данных, используемых в системе.

По числу одновременных пользователей
1. Однопользовательские ОС.
Для них характерен полный пользовательский доступ к ресурсам. Подобные системы приемлемы в основном на изолированных компьютерах.
2. Многопользовательские ОС.
Их важной компонентой являются средства защиты данных и процессов каждого пользователя, основанные на понятии владельца ресурса и на точном указании прав доступа, предоставленных каждому пользователю системы.

По аппаратной платформе (типу вычислительной техники), для которой они предназначаются, операционные системы делят на следующие группы.

- Операционные системы для смарт-карт. Некоторые из них могут управлять только одной операцией, например, электронным платежом. Некоторые смарт-карты являются JAVA-ориентированным и содержат интерпретатор виртуальной машины JAVA. Апплеты JAVA загружаются на карту и выполняются JVM-интерпретатором. Некоторые из таких карт могут одновременно управлять несколькими апплетами JAVA, что приводит к многозадачности и необходимости планирования.
- Встроенные операционные системы. Управляют карманными компьютерами (lialm OS, Windows CE – Consumer Electronics – бытовая техника), мобильными телефонами, телевизорами, микроволновыми печами и т.п.
- Операционные системы для персональных компьютеров, например, Windows, Linux, Mac OSX и др.
- Операционные системы мини-ЭВМ, например, RT-11 для PDP-11 – ОС реального времени, RSX-11 M для PDP-11 – ОС разделения времени, UNIX для PDP-7.
- Операционные системы мэйнфреймов (больших машин), например, OS/390, происходящая от OS/360 (IBM). Обычно ОС мэйнфреймов предполагает одновременно три вида обслуживания: пакетную обработку, обработку транзакций (например, работа с БД, бронирование авиабилетов, процесс работы в банках) и разделение времени.
- Серверные операционные системы, например, UNIX, Windows Server 2016, Linux Область применения – ЛВС, региональные сети, Intranet, Internet.
- Кластерные операционные системы. Кластер – слабо связанная совокупность нескольких вычислительных систем, работающих совместно для выполнения общих приложений и представляющихся пользователю единой системой, например, Windows Cluster Server, Sun Cluster (базовая ОС – Solaris).

По способу построения

1. Микроядерные
2. Монолитные

По способу загрузки

1. Загружаемые ОС (большинство).

2. системы, постоянно находящиеся в памяти вычислительной системы. Последние, как правило, специализированные и используются для управления работой специализированных устройств (например, в БЦВМ баллистической ракеты или спутника, научных приборах, автоматических устройствах различного назначения и др.).